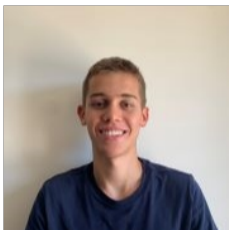
Projeto CG

Grupo TP-06

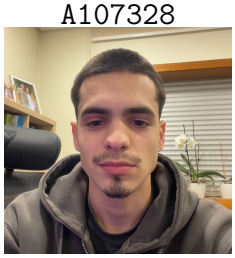
Universidade do Minho
Licenciatura em Engenharia Informática
Computação Gráfica - 2025/26

Gonçalo Costa
A107381



Diogo Costa
A107328

Lourenço Martins
A106849



Gonçalo Sá
A107376

29 de abril de 2026

Contents

1 Introdução **2**

2 Generator **2**
2.1 Construção de modelos com *patches* de Bézier 2
2.1.1 Fórmula da superfície de Bézier 2
2.1.2 Tessellação e triangulação 2

3 Engine **3**
3.1 Alterações às estruturas de dados 3
3.2 Rotações dependentes do tempo 3
3.3 Translações dirigidas por curvas de Catmull-Rom 3
3.3.1 Cálculo do ponto na curva 3
3.3.2 Parametrização global 4
3.3.3 Alinhamento com a tangente 4
3.4 Utilização de VBOs 4
3.5 Câmara esférica orbital 5

4 Sistema Solar e Extras **5**
4.1 Visão geral 5
4.2 Cometas com *patches* de Bézier 5
4.3 Extras de realismo astronómico 6
4.4 Resultado final 7

5 Testes **8**

6 Conclusão **8**

1 Introdução

O presente relatório detalha o desenvolvimento da terceira fase do projeto da unidade curricular de Computação Gráfica. Esta fase acrescentou duas grandes funcionalidades ao trabalho já realizado: a capacidade do *Generator* produzir modelos baseados em *patches* de Bézier, e a capacidade do *Engine* suportar transformações geométricas animadas e renderização com *Vertex Buffer Objects* (VBOs).

As animações assentam em dois mecanismos: rotações controladas por um período de tempo e translações dirigidas por curvas de Catmull-Rom, com suporte ao alinhamento do objeto com a tangente da curva. A cena de demonstração é um sistema solar dinâmico com todos os planetas, a Lua da Terra, os anéis de Saturno e dois cometas construídos com *patches* de Bézier.

2 Generator

2.1 Construção de modelos com *patches* de Bézier

O *Generator* foi expandido com o comando `patch`, que recebe um ficheiro `.patch`, o nível de tessellação e o ficheiro de saída. O formato do ficheiro segue a especificação fornecida: a primeira linha contém o número de *patches*; seguem-se os 16 índices de pontos de controlo de cada *patch* separados por vírgulas; depois o número total de pontos de controlo e as suas coordenadas (x, y, z).

2.1.1 Fórmula da superfície de Bézier

Cada ponto da superfície é calculado através dos polinómios de Bernstein de grau 3. Dados os parâmetros $(u, v) \in [0, 1]^2$, o ponto resultante é:

$$\mathbf{p}(u, v) = \sum_{i=0}^{3} \sum_{j=0}^{3} B_i(u)\, B_j(v)\, \mathbf{P}_{ij} \tag{1}$$

onde $\mathbf{P}_{ij}$ são os 16 pontos de controlo do *patch* e $B_k(t)$ são os polinómios de Bernstein:

$$B_0(t) = (1-t)^3, \quad B_1(t) = 3t(1-t)^2, \quad B_2(t) = 3t^2(1-t), \quad B_3(t) = t^3 \tag{2}$$

2.1.2 Tessellação e triangulação

Para cada *patch*, a superfície é amostrada numa grelha $T \times T$ (onde T é o nível de tessellação). Para cada célula (i, j) da grelha, calculam-se quatro cantos com os parâmetros:

$$\mathrm{P1} = \left(\tfrac{i}{T}, \tfrac{j}{T}\right), \quad \mathrm{P2} = \left(\tfrac{i+1}{T}, \tfrac{j}{T}\right), \quad \mathrm{P3} = \left(\tfrac{i+1}{T}, \tfrac{j+1}{T}\right), \quad \mathrm{P4} = \left(\tfrac{i}{T}, \tfrac{j+1}{T}\right) \tag{3}$$

Os quatro cantos são divididos em dois triângulos — (P1, P4, P2) e (P2, P4, P3) — respeitando a *winding order* para que as normais apontem para o exterior.

3 Engine

3.1 Alterações às estruturas de dados

Para suportar as transformações animadas foram adicionados três campos à estrutura *Transform*:

- **time** — período em segundos para uma rotação completa ou para percorrer toda a curva;
- **align** — indica se o objeto deve orientar-se com a tangente da curva (exclusivo da translação animada);
- **points** — vetor de pontos de controlo da curva de Catmull-Rom.

Foi também criada a estrutura *ModelVBO*, que armazena o identificador do *buffer* na GPU (`vbo_id`) e o número de vértices (`vert_count`), substituindo o envio de vértices em modo imediato.

3.2 Rotações dependentes do tempo

O elemento XML `<rotate>` pode agora receber o atributo `time` em vez de `angle`. Em cada *frame*, o ângulo de rotação é calculado a partir do tempo decorrido, obtido com `glutGet(GLUT_ELAPSED_TIME)`:

$$\alpha(t) = \frac{t}{t_r} \times 360 \tag{4}$$

onde t é o tempo decorrido em segundos e t_r o período definido no XML. Se o atributo `time` não estiver presente, a rotação é estática e usa o valor de `angle`.

3.3 Translações dirigidas por curvas de Catmull-Rom

O elemento `<translate>` pode agora receber os atributos `time` e `align`, e um conjunto de pontos filhos `<point>` que definem a curva. O mínimo de 4 pontos é obrigatório pela definição da curva de Catmull-Rom.

3.3.1 Cálculo do ponto na curva

A função `getCatmullRomPoint` calcula um ponto dado quatro pontos de controlo consecutivos $\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3$ e um parâmetro local $t \in [0, 1]$:

$$\mathbf{p}(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \cdot M \cdot \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}_2 \\ \mathbf{P}_3 \end{bmatrix} \tag{5}$$

onde M é a matriz de Catmull-Rom:

$$M = \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1.0 & -2.5 & 2.0 & -0.5 \\ -0.5 & 0.0 & 0.5 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \end{bmatrix} \tag{6}$$

A curva interpolada passa pelos pontos $\mathbf{P}_1$ e $\mathbf{P}_2$; os pontos $\mathbf{P}_0$ e $\mathbf{P}_3$ são auxiliares e controlam as tangentes nas extremidades do segmento, garantindo continuidade C^1 entre segmentos adjacentes — isto é, a curva é contínua e sem quebras abruptas de direção.

A derivada $\mathbf{p}'(t)$, necessária para o alinhamento, é obtida substituindo o vetor $T = [t^3\ t^2\ t\ 1]$ por $T' = [3t^2\ 2t\ 1\ 0]$ na equação (5).

3.3.2 Parametrização global

A função `getGlobalCatmullRomPoint` recebe um parâmetro global $g_t \in [0, 1[$ que representa a posição ao longo de toda a curva fechada. O parâmetro é calculado a partir do tempo decorrido como:

$$g_t = \frac{\mathrm{fmod}(t,\, t_r)}{t_r} \tag{7}$$

garantindo que g_t se repete ciclicamente e a animação nunca para. O segmento e o parâmetro local são então determinados por:

$$t_{\text{global}} = g_t \times N, \quad \text{index} = \lfloor t_{\text{global}} \rfloor, \quad t_{\text{local}} = t_{\text{global}} - \text{index} \tag{8}$$

onde N é o número de pontos de controlo. Os quatro pontos do segmento são selecionados com índices cíclicos (módulo N) para garantir a continuidade da curva fechada.

3.3.3 Alinhamento com a tangente

Quando `align="true"`, a orientação do objeto é atualizada em cada *frame* para que o seu eixo X aponte na direção da tangente. O sistema de eixos é construído como:

$$\mathbf{X}_i = \text{normalizar}(\mathbf{p}'(t))\,, \quad \mathbf{Z}_i = \text{normalizar}(\mathbf{X}_i \times \mathbf{Y}_{\text{up}})\,, \quad \mathbf{Y}_i = \text{normalizar}(\mathbf{Z}_i \times \mathbf{X}_i) \tag{9}$$

onde $\mathbf{Y}_{\text{up}} = (0, 1, 0)$ é o vetor de referência e $\times$ representa o produto vetorial. Os três vetores são organizados numa matriz de rotação 4×4 e aplicados com `glMultMatrixf` após a translação.

3.4 Utilização de VBOs

Na fase anterior, os vértices eram enviados à GPU em modo imediato em cada *frame*. Nesta fase, os dados de geometria são carregados uma única vez na inicialização pela função `loadModel`:

1. Leitura dos vértices do ficheiro `.3d` para um vetor de *floats* no formato $[x, y, z, x, y, z, \ldots]$;
2. Geração de um *buffer* com `glGenBuffers` e ativação com `glBindBuffer`;
3. Transferência dos dados para a GPU com `glBufferData(GL_STATIC_DRAW)`;
4. Armazenamento do identificador e da contagem de vértices na estrutura *ModelVBO*.

Durante a renderização, a função `drawGroup` usa `glBindBuffer`, `glVertexPointer` e `glDrawArrays(GL_TRIANGLES)` para cada modelo. O estado `GL_VERTEX_ARRAY` é ativado uma única vez no arranque com `glEnableClientState`. Uma garantia importante desta abordagem é que o carregamento dos buffers ocorre durante o *parsing* do XML, percorrendo a árvore de grupos pela mesma ordem que a renderização — assim, cada `vbo_id` armazenado na estrutura *ModelVBO* corresponde sempre ao modelo correto no momento do desenho.

3.5 Câmara esférica orbital

A câmara implementada assenta em coordenadas esféricas e foca-se num ponto alvo móvel (*lookAt*). O utilizador pode orbitar em torno desse ponto com o botão esquerdo do rato (altera os ângulos α e β), fazer *zoom* com o botão direito (altera o raio r), e deslocar o ponto alvo com as setas direcionais e as teclas `w/s`. A posição da câmara é recalculada em cada interação através das equações de conversão esférica-cartesiana:

$$\text{camX} = \text{lookX} + r\,\sin(\alpha)\cos(\beta), \quad \text{camY} = \text{lookY} + r\,\sin(\beta), \quad \text{camZ} = \text{lookZ} + r\,\cos(\alpha)\cos(\beta) \tag{10}$$

Os ângulos iniciais α e β e o raio r são calculados automaticamente a partir das posições `position` e `lookAt` lidas do XML, pelo que a câmara parte sempre da perspetiva definida na configuração. O ângulo β é restrito ao intervalo $[-85, 85]$ para evitar a inversão da perspetiva no polo.

4 Sistema Solar e Extras

4.1 Visão geral

A cena de demonstração inclui o Sol, todos os planetas do sistema solar, a Lua da Terra, os anéis de Saturno e dois cometas. Todos os movimentos orbitais e as rotações axiais são animados e dependentes do tempo. A hierarquia de grupos XML garante que os corpos dependentes (Lua, anéis de Saturno) herdam corretamente as transformações dos seus grupos pai.

No ficheiro XML, cada planeta é um grupo independente ao mesmo nível — não existem relações de pai/filho entre planetas. As únicas hierarquias reais presentes na cena são duas: a Lua como subgrupo da Terra, e os anéis de Saturno como subgrupo de Saturno. Graças ao isolamento de matrizes com `glPushMatrix`/`glPopMatrix`, cada subgrupo herda automaticamente a translação orbital do seu pai.

Figure 1: As duas hierarquias de grupos presentes na cena. À esquerda: a Lua orbita a Terra como subgrupo, herdando a sua translação orbital. À direita: os anéis de Saturno acompanham o planeta como subgrupo, com uma escala aplicada para os achatar.

4.2 Cometas com *patches* de Bézier

O enunciado requeria pelo menos um cometa construído com *patches* de Bézier e trajetória de Catmull-Rom. Foram implementados dois:

- **Cometa 1** — órbita excêntrica e inclinada relativamente ao plano XZ, com `align="true"` para que o objeto se oriente com a tangente em cada instante. A sua geometria é definida por `comet.patch`, composto por 6 *patches* de Bézier. Os 3 primeiros formam a superfície lateral cônica: os 4 pontos de controlo do topo convergem todos para o mesmo ponto $(0, 0, 3)$, forçando a superfície a afunilar numa ponta, enquanto a base se abre para um raio de 1 unidade em $z = 0$. Os 3 *patches* restantes fecham a superfície pelo lado da base, sendo construídos com os índices dos primeiros transpostos para inverter a *winding order* e garantir normais coerentes em toda a superfície.

- **Cometa 2** — órbita circular de raio muito superior ao dos planetas exteriores, simulando um cometa de longo período que percorre regiões afastadas do sistema solar.

4.3 Extras de realismo astronómico

Para além dos requisitos mínimos, foram introduzidos vários detalhes de precisão astronómica:

- **Urano com inclinação axial de ∼98°** — O eixo de rotação foi definido como quase horizontal (`x="0.99" y="0.1" z="0"`), fazendo Urano rolar ao longo da sua órbita, tal como acontece na realidade.

- **Vénus com rotação retrógrada** — O sentido de rotação foi invertido (`y="-1"`), reproduzindo a rotação retrógrada de Vénus. O período de 60 segundos reflete também o seu dia solar extremamente longo.

- **Mercúrio com órbita excêntrica** — A curva de Catmull-Rom de Mercúrio é assimétrica: o afélio está a $x = 24$ e o periélio a $x = -18$, gerando uma elipse achatada que simula a maior excentricidade orbital do sistema solar ($e \approx 0.21$).

- **Lua com inclinação orbital** — Os pontos de controlo da órbita da Lua têm coordenadas y não nulas ($y = \pm 0.5$ em pontos alternados), reproduzindo a inclinação de ∼5° da órbita lunar em relação à eclíptica.

- **Anéis de Saturno em hierarquia correta** — O torus dos anéis é grupo filho de Saturno, acompanhando-o automaticamente na sua órbita. A escala $(5.5, 0.15, 5.5)$ achata o torus para simular a geometria real dos anéis.

4.4 Resultado final

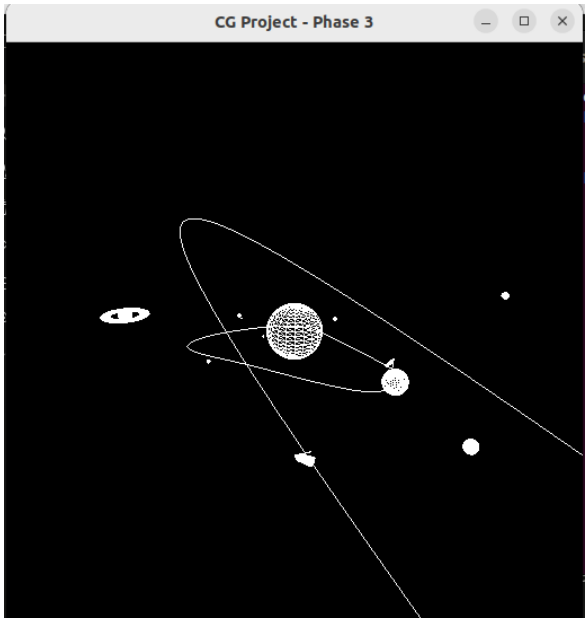


Figure 2: Visão global do sistema solar animado em modo *wireframe*.

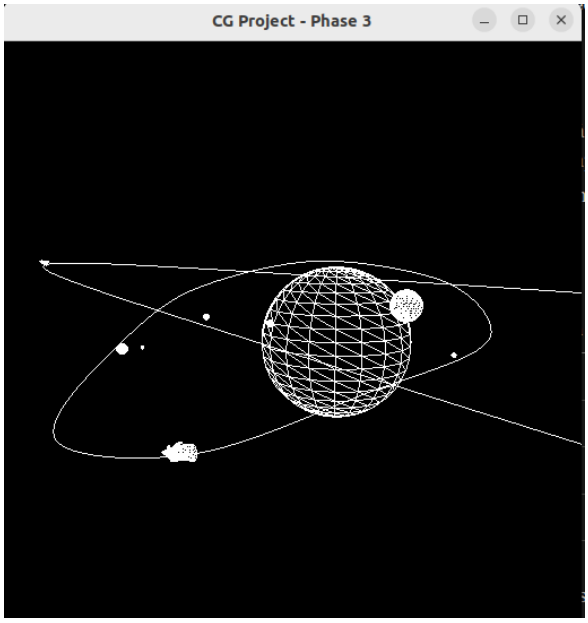


Figure 3: Sistema solar com as trajetórias dos dois cometas visíveis.

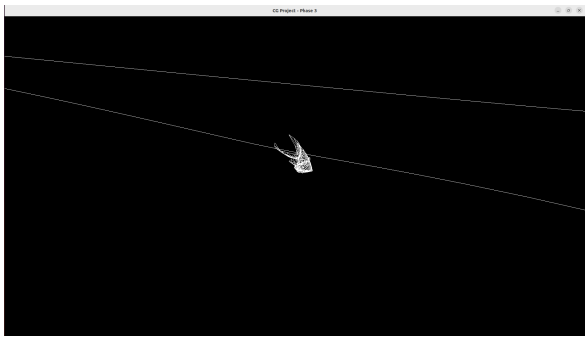


Figure 4: Detalhe do cometa 1 construído com *patches* de Bézier, alinhado com a tangente da curva de Catmull-Rom.

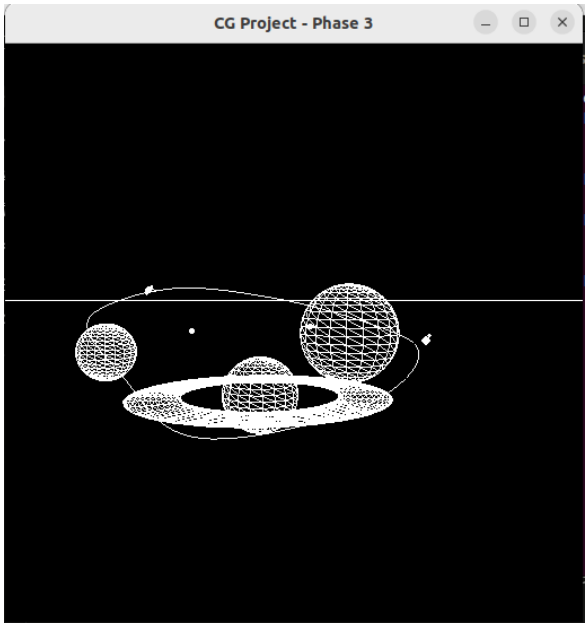


Figure 5: Detalhe de Saturno com os anéis (torus) a acompanhar o planeta na sua órbita.

5 Testes

Antes da elaboração da cena final, a correção das funcionalidades implementadas foi validada com dois cenários de teste fornecidos.

O **Teste 1** valida em simultâneo a translação animada por curva de Catmull-Rom com alinhamento e a rotação animada em grupos hierárquicos. O ficheiro XML define um grupo pai com rotação animada (`time="3"` em torno do eixo Y) e um grupo filho cujo modelo (`bezier_10.3d`) percorre uma curva fechada de 4 pontos com `align="true"`, uma rotação estática de -90 no eixo X e uma escala de 0.5. O resultado esperado é o teapot a mover-se ao longo da órbita orientando-se com a tangente da curva, enquanto toda a cena roda em torno da origem.

O **Teste 2** valida a geração de modelos com *patches* de Bézier e a sua renderização com VBOs. O ficheiro XML carrega o modelo `bezier_10.3d` (gerado a partir de `teapot.patch` com tessellação 10) com uma rotação estática de -90 no eixo X. O resultado esperado é o teapot estático corretamente posicionado e com a geometria bem formada.

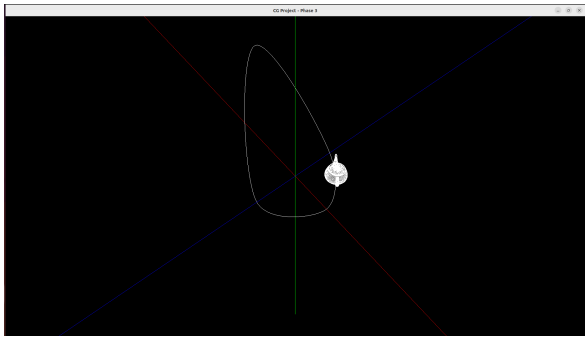


Figure 6: Teste 1: teapot a percorrer a curva de Catmull-Rom com `align="true"`, dentro de um grupo com rotação animada.

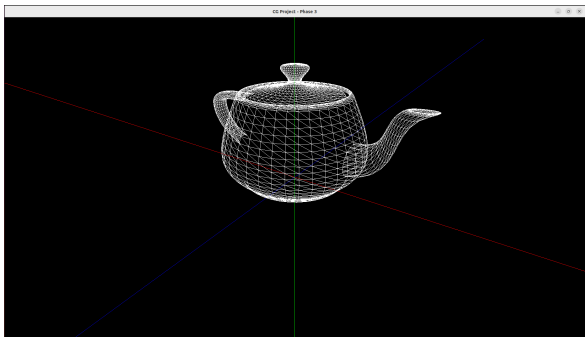


Figure 7: Teste 2: teapot gerado a partir de *patches* de Bézier com tessellação 10, renderizado com VBOs.

6 Conclusão

Ao longo desta terceira fase, foram implementados com sucesso todos os requisitos do enunciado: geração de modelos com *patches* de Bézier, translações animadas por curvas de Catmull-Rom com suporte a alinhamento, rotações dependentes do tempo e renderização com VBOs.

A migração para VBOs elimina o reenvio de vértices em cada *frame*, transferindo os dados de geometria para a memória da GPU durante a inicialização, com ganhos de desempenho proporcionais ao número de polígonos da cena. A curva de Catmull-Rom permitiu criar trajetórias suaves e contínuas para os corpos celestes, e os *patches* de Bézier abriram a possibilidade de modelar formas orgânicas de forma paramétrica.

Para além dos requisitos mínimos, foram introduzidos detalhes de realismo astronómico que enriquecem a cena: a inclinação axial de Urano, a rotação retrógrada de Vénus, a órbita excêntrica de Mercúrio, a inclinação da órbita da Lua e dois cometas com geometrias e órbitas distintas. O grupo considera o trabalho desta fase concluído com sucesso e a base preparada para a fase seguinte, que introduzirá iluminação, normais e texturas.